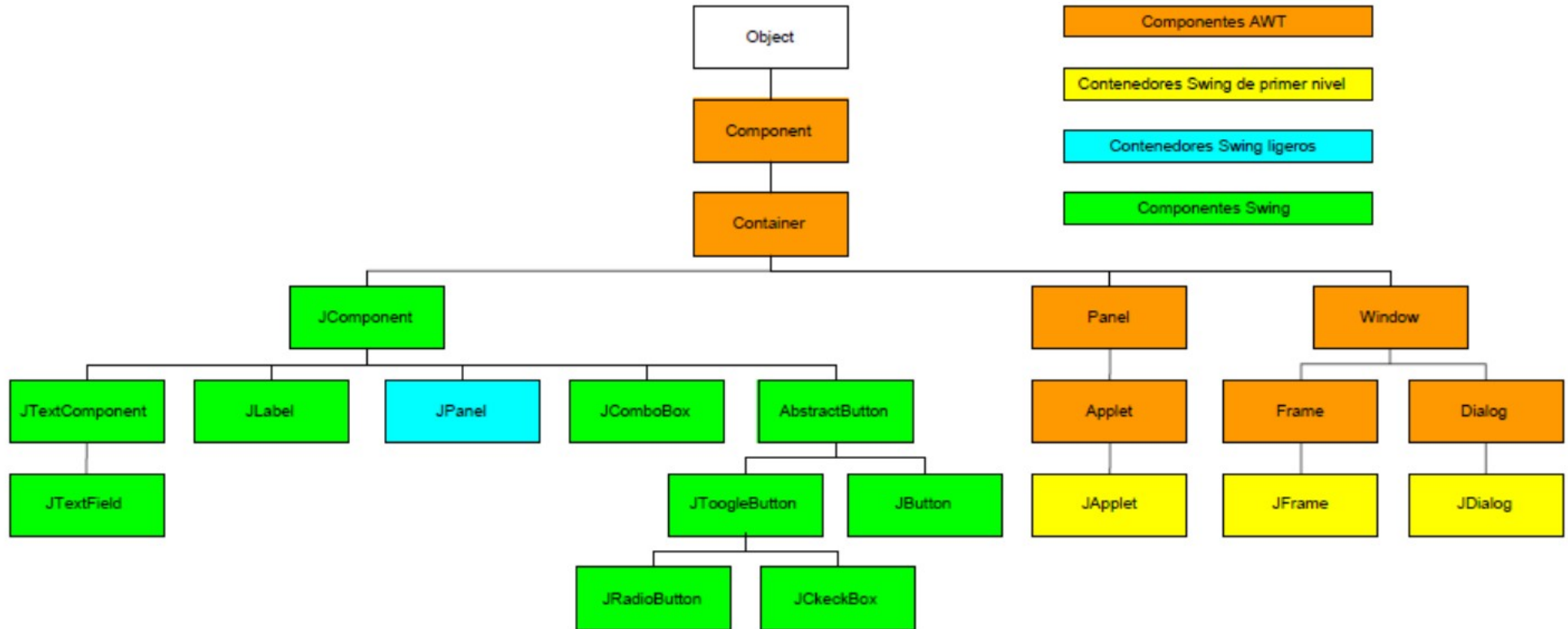
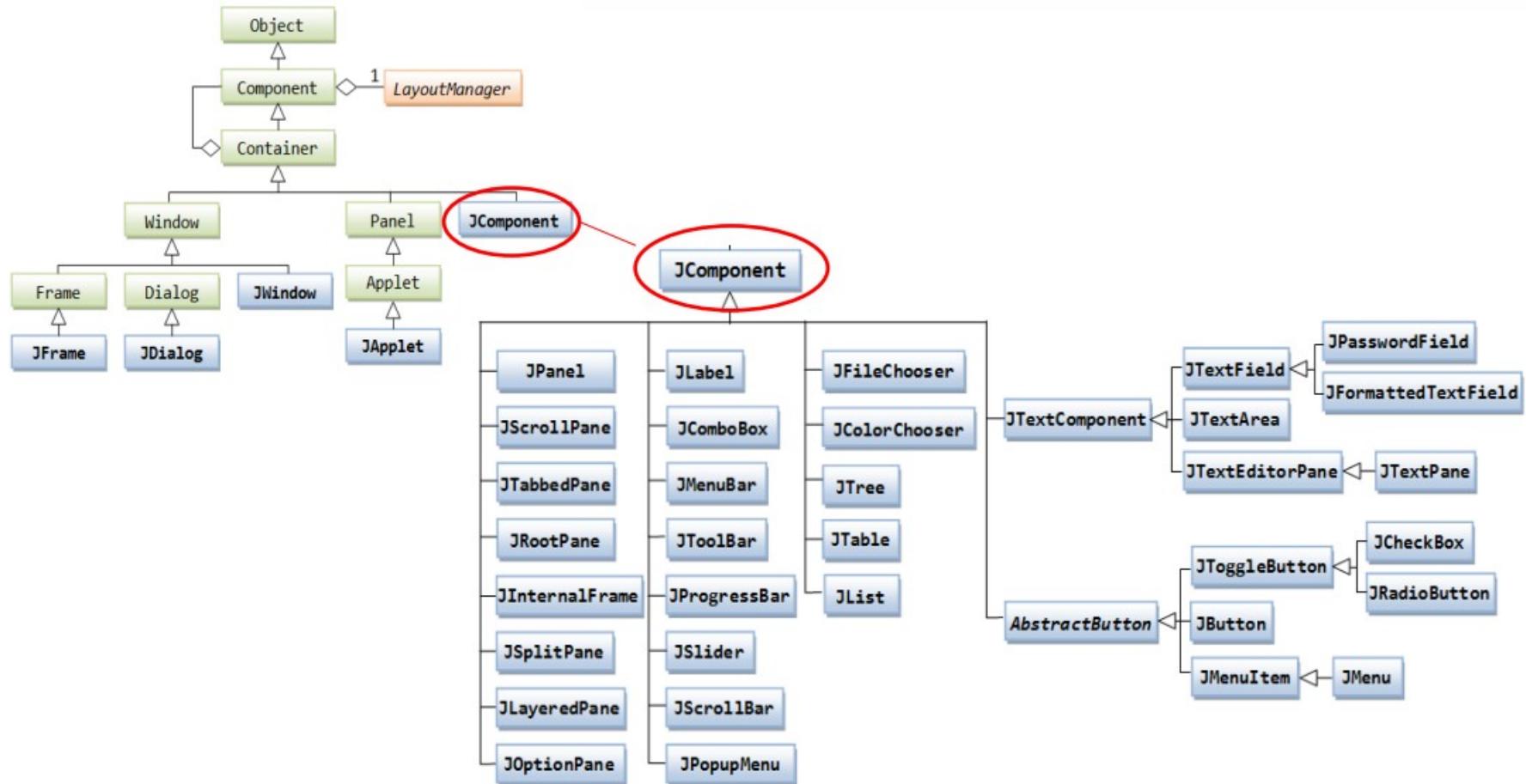


Resumen controles Swing



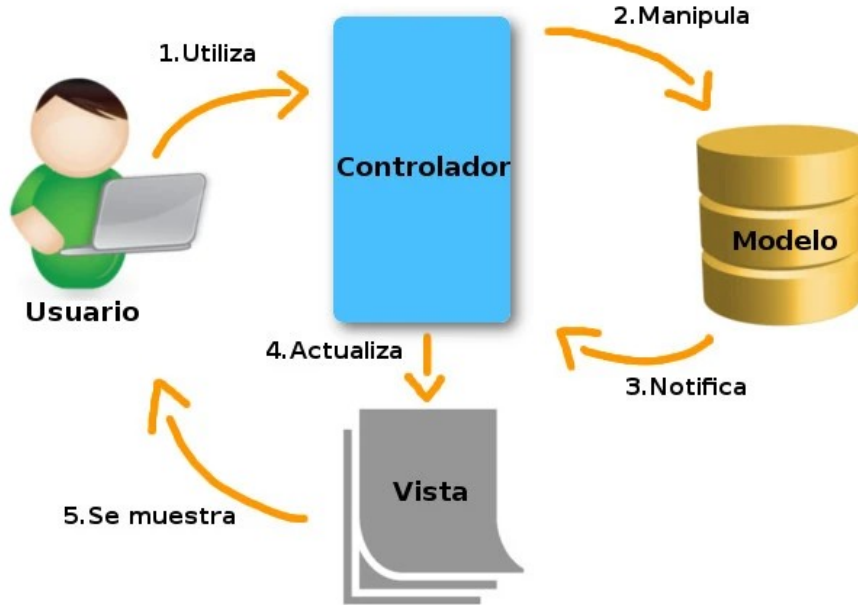
Resumen controles Swing



Modelo Vista Controlador (MVC)

- Estilo de arquitectura software que separa la aplicación en 3 capas diferenciadas: los **datos**, la **interfaz** y la **lógica** de control, en una aplicación. Facilita cambios (SGBD...) y mantenimiento (funciones).
- Empleado frecuentemente en **web**:
 - **Vista**: código HTML, CSS
 - **Modelo**: base de datos y lógica de negocio (MySQL, PHP)
 - **Controlador**: recibe y gestiona eventos de la vista (PHP, JS, JSP...)
- También en aplicaciones de escritorio.
 - **Vista**: interfaz gráfica (Swing, JavaFX...)
 - **Modelo**: base de datos y diseño de clases/objetos (clase Persona).
 - **Controlador**: lógica que comunica la vista con el modelo (gestión de eventos...)

Modelo Vista Controlador (MVC)



Vista: ventana principal de la aplicación.

Sólo contiene código referente a la construcción y organización de los componentes gráficos. No hay lógica ni modelo de datos, al ser independiente se facilita su reutilización.

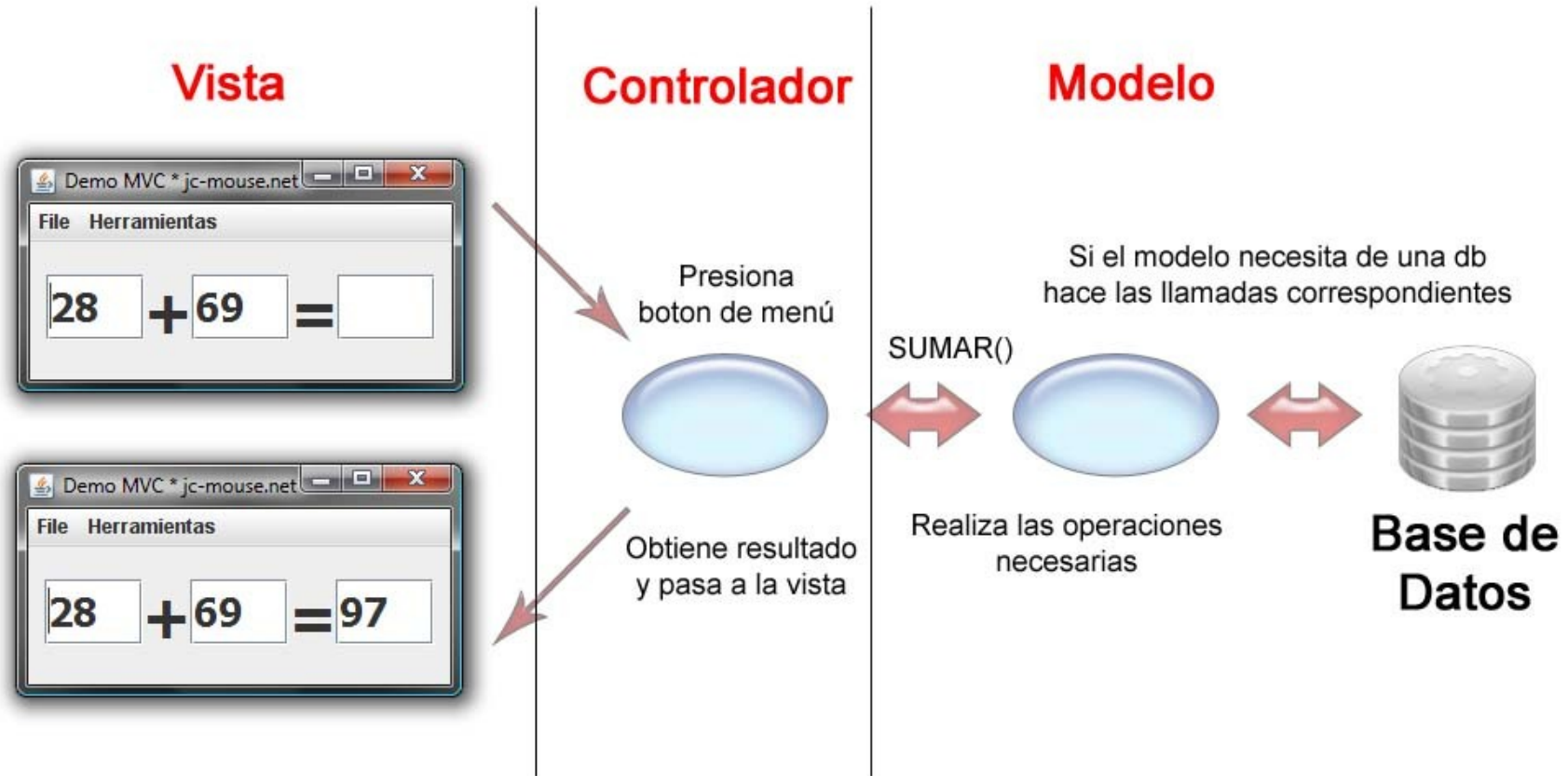
Modelo: datos de la aplicación y su gestión, lógica de negocio.

Operaciones: alta elementos, eliminar, buscar, guardar, cargar, etc. Independiente de la vista y del controlador.

Controlador: capa que comunica a las otras dos.

Tiene implementados los manejadores de eventos, también es quien ejecuta las operaciones del modelo en respuesta a esos eventos.

Modelo Vista Controlador (MVC)



Frameworks MVC por lenguaje

- Java: Struts / Spring
- PHP: CodeIgniter
- Ruby: Ruby on Rails
- .NET: ASP.NET
- JavaScript: AngularJS
- Python: Django

Modelo Vista Controlador (MVC)

Implementación

Creamos un nuevo proyecto en Netbeans.

Creamos una estructura de paquetes (Controller, Model, View), fundamental para separar cada componente y ser más organizados. Veamos cómo debe quedar:



Como se puede observar, mantenemos el paquete *default* junto al *Main.java* que nos crea Netbeans, este *main* es el que servirá como nuestro *index.php/index.html* de la aplicación, nuestro "lanzador" de la misma.

Modelo Vista Controlador (MVC)

Implementación

Creamos la lógica de la aplicación. Nueva clase en el paquete Model, llamado "modelo.java":

```
package model;
/**
 * @web http://jc-mouse.blogspot.com/
 * @author Mouse
 * En esta clase se guarda la lógica del sistema, para
 * consiste en una SUMA
 */
public class modelo {
    //Variables
    private int valor1=28;
    private int valor2=69;
    private int total = sumar();
}
```

```
public modelo(){}

public void set_valor1(int val){
    this.valor1=val;
}

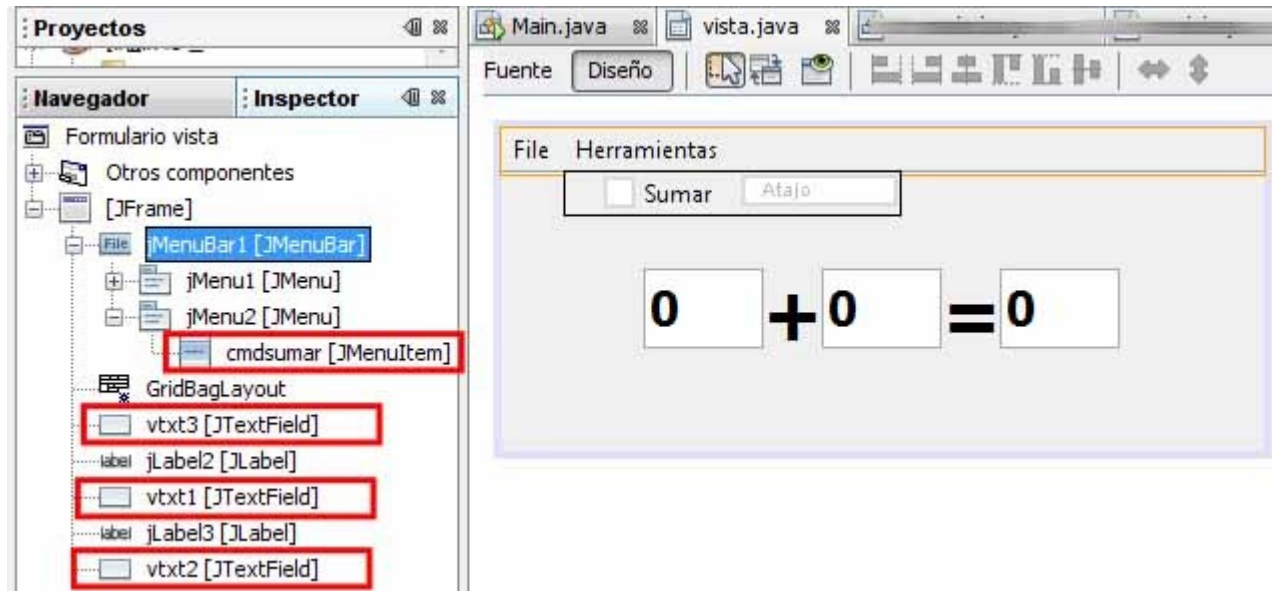
public int get_valor1(){
    return this.valor1;
}

public void set_valor2(int val){
    this.valor2=val;
}
```


Modelo Vista Controlador (MVC)

Implementación

Diseñemos ahora la interfaz de usuario, nuestra VISTA. Añadimos un JFrame al paquete VIEW ("vista.java"). En rojo mostramos los componentes que tendrán interacción con el modelo. Para evitar confusiones, es mejor renombrarlos como se ve abajo:

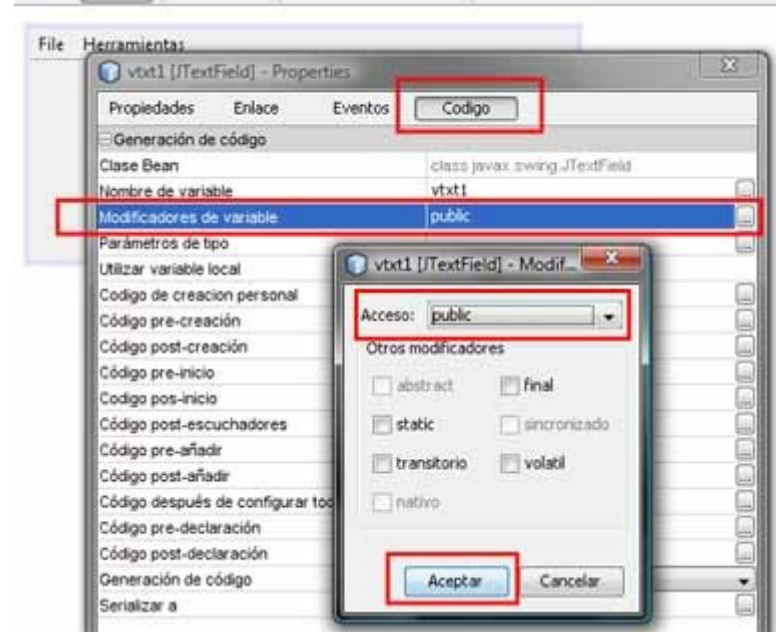


Modelo Vista Controlador (MVC)

Netbeans, al añadir objetos al JFrame, los marca como PRIVATE. Debemos cambiar esto, para ello seleccionamos el objeto "vtxt1" y en sus propiedades, en la pestaña CÓDIGO, elegimos el campo MODIFICADORES DE VARIABLE.

En la ventana que se muestra, cambiamos el atributo PRIVATE, por PUBLIC y aceptamos.

Hay que repetir esta acción para todos los objetos que están marcados en rojo en la imagen anterior.



Modelo Vista Controlador (MVC)

Ahora continuamos con el CONTROLADOR de nuestra aplicación. Creamos una clase "controlador.java" en el paquete CONTROLLER con el código:

```
package controller;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import model.modelo;
import view.vista;

/**
 * @web http://jc-mouse.blogspot.com/
 * @author Mouse
 */
public class controlador implements ActionListener{

    private vista vista;
    private modelo modelo;
```

```
//En el constructor inicializamos nuestros objetos y tambien
//añadimos el ActionListener al boton "cmdsumar" de la VISTA
public controlador( vista vista , modelo modelo){
    this.vista = vista;
    this.modelo = modelo;
    this.vista.cmdsumar.addActionListener(this);
}

//Inicia los valores del JFrame VISTA con los datos del MODELO
public void iniciar_vista(){
    vista.setTitle( "Demo MVC * jc-mouse.net" );
    vista.setLocationRelativeTo(null);
    vista.vtxt1.setText( String.valueOf(modelo.get_valor1()) );
    vista.vtxt2.setText( String.valueOf(modelo.get_valor2()) );
    vista.vtxt3.setText( String.valueOf(modelo.get_total()) );
}
```

Modelo Vista Controlador (MVC)

Explicación

La clase “controlador” implementa el ***ActionListener***. Esto permite responder desde esta clase a los eventos realizados desde la interfaz (VISTA), además en el constructor de clase se pasan como parámetros la clase VISTA y la clase MODELO.

La clase, además, cuenta las funciones:

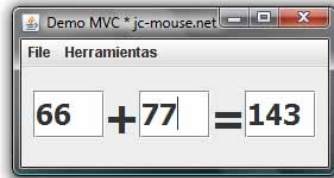
- INICIAR(): inicializa los valores de la interfaz, como ser el atributo titulo del JFrame, posicionamiento en pantalla, valores iniciales de los jtextbox, etc.
- También tenemos un método “action performed”: captura el evento realizado desde la interfaz. En este ejemplo es solo un CLICK en el botón de menú CMDSUMAR. Se obtienen los datos correspondientes e invoca al modelo para procesar la información y obtener una respuesta.

La interfaz, para llegar al modelo, pasa por el controlador y viceversa.

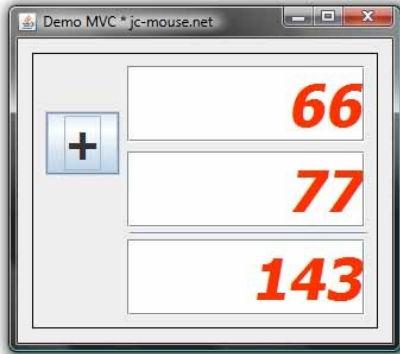
Modelo Vista Controlador (MVC)

Finalmente debemos implementar nuestro main.java:

interfaz 1



interfaz 2



```
package jc_mvc_demo;

import controller.controlador;
import model.modelo;
import view.vista;

/**
 * @web http://jc-mouse.blogspot.com/
 * @author Mouse
 */
public class Main {

    public static void main(String[] args) {

        //nuevas instancias de clase
        modelo modelo = new modelo();
        vista vista = new vista();
        controlador controlador = new controlador( vista , modelo );
        //se inicia la vista
        controlador.iniciar_vista();

        vista.setVisible(true);

    }
}
```